Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Florianópolis
2027

Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Universidade Federal de Santa Catarina

Orientador: Antônio Augusto Fröhlich

Florianópolis
2027

LISTA DE ILUSTRAÇÕES

Figura 1 – Hipervisor Tipo 1. 13
Figura 2 – Hipervisor Tipo 2. 13
Figura 3 – Níveis de Privilégio RISC-V. 15

Figura 4 – Fluxo Geral de Inicialização do Hipervisor. 21
Figura 5 – Fluxo Geral de Inicialização da Arquitetura RISC-V. 22
Figura 6 – Fluxo Geral de Inicialização do Kernel. 23
Figura 7 – Diagrama de Classe: VirtualMachine. 24
Figura 8 – Diagrama de Classe: GenericVirtualMachine. 25
Figura 9 – Fluxo Geral de Inicialização de uma *GenericVirtualMachine*. 25
Figura 10 – Diagrama de classe da *VirtualCPU*. 26
Figura 11 – Fluxo Geral de Inicialização de uma *VirtualCPU* RISC-V. 28
Figura 12 – Troca de Contexto Entre Duas VMs 29
Figura 13 – Fluxo de Tratamento de um *Load Access Fault* - RISC-V. 31
Figura 14 – Fluxo de Tratamento de um *Store Access Fault* – RISC-V. 32

Figura 15 – Tempos de Execução Das Iterações de Uma VM Com Carga em CPU - Sem Linux. 36
Figura 16 – Tempos de Execução Das Iterações de Uma VM Com Carga em Memória - Sem Linux. 37
Figura 17 – Tempos de Execução Das Iterações de Uma VM Com Carga em CPU - Com Linux. 37
Figura 18 – Tempos de Execução Das Iterações de Uma VM Com Carga em Memória - Com Linux. 38
Figura 19 – Tempos de Execução Das Iterações de Uma VM Com Carga em CPU - Com Linux Invalidando Cache. 38
Figura 20 – Tempos de Execução Das Iterações de Uma VM Com Carga em Memória - Com Linux Invalidando Cache. 39

SUMÁRIO

1 INTRODUÇÃO 5
1.1 Objetivos 6
1.1.1 Objetivos Gerais 6
1.1.2 Objetivos Específicos 6

2 FUNDAMENTAÇÃO TEÓRICA 7
2.1 Sistemas Embarcados 7
2.2 Sistemas Operacionais 7
2.3 Sistemas Operacionais de Tempo Real 7
2.4 EPOS 8
2.5 Interrupções 8
2.5.1 Interrupções: RISC-V 8
2.6 Exceções 9
2.6.1 Exceções: RISC-V 9
2.7 Driver de Dispositivo 10
2.8 Virtualização 10
2.9 VirtIO 11
2.10 Diferenças Entre Virtualização e Emulação 12
2.11 Hipervisors 12
2.11.1 Tipo 1 - Bare Metal 12
2.11.2 Tipo 2 13
2.12 Interceptação e Emulação 14
2.13 RISC-V 14
2.13.1 Extensões 14
2.13.2 Níveis de Privilégio 15
2.13.3 Registradores de Controle 16
2.14 Preempção e Escalonamento 16
2.14.1 Escalonamento Involuntário 16
2.14.2 Escalonamento Voluntário 17

3 MATERIAIS UTILIZADOS 18

4 METODOLOGIA 19
4.1 Virtualização Baseada Machine Mode 19

5 IMPLEMENTAÇÃO 20

5.1 Estrutura de Pastas . . . 20
5.2 Inicialização . . . 21
5.2.1 RISC-V . . . 21
5.2.2 Máquina . . . 22
5.2.3 Kernel . . . 22
5.3 Máquinas Virtuais . . . 23
5.3.1 Inicialização . . . 24
5.3.2 *GenericVirtualMachine* . . . 24
5.3.2.1 Inicialização . . . 25
5.4 CPUs Virtuais . . . 25
5.4.1 RISC-V . . . 26
5.4.1.1 Virtualização de Interrupções Temporais . . . 26
5.4.1.2 Virtualização de Interrupções Externas . . . 27
5.4.1.3 Inicialização . . . 27
5.5 Troca de Contexto Entre Máquinas Virtuais . . . 28
5.5.1 RISC-V . . . 28
5.6 Camada de Emulação . . . 30
5.6.1 RISC-V . . . 30
5.6.1.1 *Illegal Instruction* . . . 30
5.6.1.2 *Load Access Fault* . . . 30
5.6.1.3 *Store Access Fault* . . . 31
5.7 Detecção e Tratamento de VMs em Idle na RISC-V . . . 32

6 RESULTADOS . . . 34
6.1 Primeiro Experimento - Interferência Via Cache . . . 34

REFERÊNCIAS . . . 40

1 INTRODUÇÃO

Sistemas embarcados estão presentes em uma ampla gama de aplicações, que vão desde dispositivos médicos e sistemas de automação industrial até plataformas automotivas e missões espaciais. Diferentemente da computação de propósito geral, esses sistemas operam sob rigorosos requisitos de tempo real, nos quais a corretude do comportamento não depende apenas da precisão lógica dos resultados produzidos, mas também do momento exato em que esses resultados são disponibilizados. Em muitos casos, atrasos mínimos ou falhas temporais podem comprometer a segurança, a confiabilidade e até mesmo a integridade física de pessoas e equipamentos.

Tradicionalmente, a garantia desse determinismo esteve associada ao uso de plataformas dedicadas, nas quais Sistemas Operacionais de Tempo Real (RTOSs) são empregados para assegurar previsibilidade temporal, isolamento entre tarefas e cumprimento de restrições rígidas de escalonamento. Entretanto, a evolução do software contemporâneo deslocou grande parte dos ecossistemas de desenvolvimento — bibliotecas, frameworks, ferramentas e serviços — para ambientes baseados em Sistemas Operacionais de Propósito Geral (GPOSs) - especialmente Linux. Embora esses sistemas ofereçam grande riqueza funcional e amplo suporte da comunidade, sua adoção em contextos críticos embarcados demanda esforços adicionais de adaptação, validação e integração.

Como resultado, cresce a necessidade de consolidar, em uma mesma plataforma computacional, cargas de trabalho com requisitos distintos e frequentemente conflitantes: de um lado, aplicações críticas, sensíveis a prazos temporais e dependentes de comportamento determinístico; de outro, aplicações complexas, orientadas a funcionalidades avançadas e dependentes de ambientes generalistas.

Nesse cenário, técnicas de virtualização surgem como uma alternativa promissora, pois permitem a coexistência de múltiplos ambientes de execução sobre o mesmo hardware. Dessa forma, torna-se possível executar aplicações críticas e não críticas de maneira controlada, reduzindo custos de integração e aumentando o aproveitamento da plataforma.

Paralelamente, a arquitetura aberta RISC-V vem ganhando relevância devido à sua flexibilidade, modularidade e independência tecnológica. Seu conjunto de instruções extensível, aliado à ausência de barreiras proprietárias, favorece tanto a pesquisa acadêmica quanto o desenvolvimento industrial, especialmente em soluções customizadas e de baixo custo.

Por isso, este trabalho propõe o projeto e a implementação, utilizando a linguagem C++ orientada a objetos de baixo overhead, de uma infraestrutura de virtualização para sistemas baseados em RISC-V. O foco principal reside na previsibilidade temporal, no isolamento rígido entre cargas de trabalho e no baixo custo de execução. Busca-se, assim,

contribuir para a viabilização de plataformas embarcadas capazes de integrar aplicações heterogêneas sem comprometer requisitos críticos de desempenho e confiabilidade.

1.1 OBJETIVOS

1.1.1 Objetivos Gerais

Projetar, implementar e validar uma infraestrutura de virtualização de baixo overhead para sistemas embarcados baseados na arquitetura RISC-V, utilizando a linguagem C++, com foco no isolamento de cargas de trabalho, no particionamento estático de recursos e na previsibilidade temporal necessária à execução de aplicações de tempo real.

1.1.2 Objetivos Específicos

1. Definir a arquitetura de software do hipervisor, estabelecendo os mecanismos necessários para o gerenciamento e isolamento das máquinas virtuais.

2. Implementar o núcleo de virtualização responsável pela criação, inicialização, gerenciamento e execução das máquinas virtuais em plataformas RISC-V.

3. Desenvolver mecanismos de virtualização de interrupções e exceções, assegurando o tratamento correto e eficiente dos eventos entre hardware, hipervisor e sistemas convidados.

4. Implementar mecanismos de particionamento estático e isolamento de recursos computacionais, incluindo processadores, memória e dispositivos de entrada e saída.

5. Garantir a execução isolada de aplicações com diferentes requisitos operacionais, possibilitando a coexistência de cargas de trabalho críticas e não críticas em uma mesma plataforma computacional.

6. Validar funcionalmente a infraestrutura desenvolvida por meio da execução de sistemas convidados representativos.

7. Avaliar o impacto da virtualização sobre métricas relacionadas ao desempenho e à previsibilidade temporal, incluindo latência, tempo de resposta e sobrecarga de execução.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 SISTEMAS EMBARCADOS

Sistemas embarcados são sistemas computacionais desenvolvidos para executar funções específicas dentro de dispositivos ou equipamentos maiores. Diferentemente dos computadores de propósito geral, que suportam uma ampla variedade de aplicações, os sistemas embarcados são projetados para atender requisitos particulares de uma determinada aplicação.

Uma característica fundamental desses sistemas é a estreita integração entre hardware e software, permitindo que os recursos disponíveis sejam utilizados de forma eficiente. Em geral, sistemas embarcados operam sob restrições de processamento, memória, armazenamento e energia, o que exige soluções altamente otimizadas.

2.2 SISTEMAS OPERACIONAIS

Um sistema operacional é um software responsável por gerenciar os recursos de hardware de um sistema computacional e fornecer serviços para a execução de aplicações. Atuando como uma camada de abstração entre o hardware e os programas, o sistema operacional simplifica o desenvolvimento de software ao oferecer mecanismos padronizados para acessar o processador, memória, dispositivos de entrada e saída, dispositivos armazenamento de dados, etc.

A evolução dos sistemas operacionais permitiu o surgimento de plataformas cada vez mais complexas, capazes de executar múltiplas aplicações simultaneamente e fornecer diversos serviços ao usuário. Entretanto, dependendo dos requisitos da aplicação, características como previsibilidade temporal, confiabilidade e isolamento podem se tornar tão importantes quanto o desempenho, motivando o desenvolvimento de sistemas operacionais especializados.

2.3 SISTEMAS OPERACIONAIS DE TEMPO REAL

RTOSs são sistemas desenvolvidos para aplicações em que o cumprimento de restrições temporais é tão importante quanto a corretude dos resultados produzidos. Diferentemente dos sistemas operacionais de propósito geral, que priorizam desempenho, os RTOS são projetados para garantir que tarefas críticas sejam executadas dentro de prazos previamente definidos, uma vez que o não cumprimento desses prazos pode comprometer o funcionamento do sistema.

2.4 EPOS

Embedded Parallel Operating System (EPOS) Fröhlich e Schröeder-Preikschat (1999) é um RTOS orientado a objetos desenvolvido em C++ com foco em aplicações embarcadas e sistemas dedicados. Sua arquitetura é baseada em componentes modulares, permitindo que apenas os recursos necessários para cada aplicação sejam incluídos no sistema final, reduzindo *overheads* desnecessários.

O EPOS adota o conceito de *Application-Oriented Operating System*, no qual um sistema operacional é configurado de acordo com os requisitos específicos de uma aplicação. Para isso, são utilizadas técnicas de orientação a objetos e metaprogramação em toda a base de código desse sistema.

O trabalho desenvolvido neste documento baseia-se fortemente nos princípios arquiteturais do EPOS, especialmente em sua modularidade e abstrações.

2.5 INTERRUPÇÕES

As interrupções constituem-se em um mecanismo fundamental de controle em arquiteturas computacionais, permitindo que o processador suspenda temporariamente a execução de uma tarefa para atender a eventos de alta prioridade. Este paradigma é essencial para a operação assíncrona de sistemas modernos, viabilizando a reação imediata a sinais de hardware, periféricos ou condições de erro.

Quando uma interrupção é disparada, o fluxo de execução é desviado para uma rotina especializada denominada *Interrupt Service Routine* (ISR). O tratamento ocorre geralmente em três etapas:

1. Salvamento de Contexto: o estado atual da CPU é salvo na pilha ou em uma estrutura de dados específica;

2. Execução da ISR: o processador executa o código responsável por tratar o evento que causou a interrupção;

3. Restauração do Contexto: o estado anterior é restaurado, permitindo que a tarefa originalmente interrompida retome seu processamento a partir do ponto em que foi suspensa;

2.5.1 Interrupções: RISC-V

Na arquitetura RISC-V, o gerenciamento de interrupções é estruturado de forma nativa e classificado em três categorias principais:

1. Interrupção Temporizada: originada por um temporizador - periférico padronizado Core Local Interrupter (CLINT). É comumente programada para gerar eventos periódicos;

2. Interrupção de Software: disparada programaticamente por meio da escrita em registradores específicos. É frequentemente utilizada pelo próprio Sistema Operacional (SO) para possibilitar *Inter-Processor Interrupt*s (IPIs);

3. Interrupção Externa: gerada por periféricos independentes do núcleo. Essas interrupções são tipicamente centralizadas e roteadas para o processador por meio de um controlador de interrupções - Platform-Level Interrupt Controller (PLIC).

2.6 EXCEÇÕES

Exceções são mecanismos de controle semelhantes às interrupções, pois ambos permitem que o processador suspenda temporariamente o fluxo normal de execução para tratar eventos específicos. No entanto, enquanto as interrupções são geralmente causadas por eventos assíncronos externos à CPU — como sinais de periféricos ou temporizadores — as exceções são geradas de forma síncrona pelo processador durante a execução de uma instrução.

Este mecanismo é fundamental para o tratamento de condições anômalas, tais como divisões por zero, acessos inválidos à memória, estouro de pilha ou execução de instruções não suportadas.

Ao detectar uma exceção, o processador desvia o fluxo para uma rotina de tratamento - geralmente a mesma das interrupções. A principal diferença operacional reside no pós-processamento: diferentemente das interrupções, que usualmente retomam o fluxo da tarefa original, o SO pode decidir encerrar a aplicação que causou a exceção caso ela seja classificada como fatal.

2.6.1 Exceções: RISC-V

Na arquitetura RISC-V, uma exceção pode mapear qualquer um dos os seguintes eventos:

1. *Instruction Address Misaligned*

2. *Instruction Access Fault*

3. *Illegal Instruction*

4. *Breakpoint*

5. *Load Address Misaligned*

6. *Load Access Fault*
7. *Store/AMO Address Misaligned*
8. *Store/AMO Access Fault*
9. *Environment Call from U-mode*
10. *Environment Call from S-mode*
11. *Environment Call from M-mode*
12. *Instruction Page Fault*
13. *Load Page Fault*
14. *Store/AMO Page Fault*

2.7 DRIVER DE DISPOSITIVO

Um driver é um programa responsável por permitir a comunicação entre o SO e um componente específico de hardware. Ele atua como uma camada intermediária que traduz comandos de alto nível em instruções compreensíveis pelo dispositivo, possibilitando sua utilização de forma padronizada.

Os drivers são essenciais porque cada dispositivo possui características e protocolos de comunicação próprios. Dessa forma, eles abstraem os detalhes de implementação do hardware, simplificando seu uso pelas aplicações e pelos demais componentes do sistema.

2.8 VIRTUALIZAÇÃO

Virtualização é uma técnica computacional que permite a execução simultânea e isolada de múltiplos SOs — denominados sistemas convidados — em um mesmo hardware físico. Enquanto um SO convencional atua como uma interface entre o usuário e a máquina, abstraindo a complexidade da arquitetura subjacente, existem sistemas projetados para estender essa abstração a outros SOs. Esses sistemas são denominados hipervisores, e o mecanismo que viabiliza esse processo é chamado de virtualização.

De modo geral, existem dois principais tipos de virtualização: a virtualização completa e a paravirtualização.

Na virtualização completa, também conhecida como *full virtualization*, o SO convidado executa sem ter conhecimento explícito de que está em um ambiente virtualizado. Nesse cenário, operações privilegiadas são interceptadas pelo hipervisor por meio de exceções e tratadas por mecanismos como *trap-and-emulate*. Entretanto, esse processo

pode comprometer o desempenho do sistema, devido ao custo associado às operações de salvamento e restauração de contexto realizadas a cada exceção gerada.

Em contraste, a paravirtualização adota uma abordagem cooperativa entre o SO convidado e o seu hipervisor. Nesse modelo, o sistema convidado é modificado para reconhecer a presença do ambiente virtualizado e, em vez de executar diretamente instruções privilegiadas, realiza chamadas explícitas ao hipervisor, conhecidas como *hypercalls*. Essa estratégia reduz a necessidade de exceções, diminuindo o custo associado ao processo de virtualização.

2.9 *VIRTIO*

VirtIO (RUSSELL, 2008) é um padrão para drivers de dispositivos paravirtualizados, projetado para unificar e otimizar a interface de comunicação entre os sistemas convidados e o hipervisor. Em vez de emular hardwares legados complexos e específicos, o hipervisor expõe uma série de dispositivos virtuais padronizados, permitindo que o SO convidado acesse recursos de rede, armazenamento e console de maneira eficiente e com baixa sobrecarga.

Para viabilizar essa comunicação, o modelo utiliza as chamadas *Virtqueues* — filas circulares compartilhadas responsáveis pelo envio e recebimento das requisições. Dependendo das operações suportadas, cada dispositivo virtual pode alocar uma ou mais dessas filas. Internamente, uma *Virtqueue* divide-se em três componentes principais: uma tabela de descritores (*Descriptor Table*), um anel de índices disponíveis (*Available Ring*) e um anel de índices já utilizados (*Used Ring*).

Na prática, o ciclo de vida de uma operação de Entrada e Saída (IO) ocorre em duas etapas complementares. Inicialmente, quando o sistema convidado deseja realizar uma operação, o dispositivo *VirtIO* escreve os dados na memória compartilhada, insere o descritor correspondente no *Available Ring* e notifica o hipervisor sobre a nova pendência.

Em resposta, o hipervisor consome os descritores desse anel, processa a operação solicitada e registra o resultado no *Used Ring*. Por fim, uma interrupção é enviada ao sistema convidado para sinalizar a conclusão da tarefa. Graças a esse design, múltiplas requisições podem ser tratadas de maneira assíncrona e em lote, o que minimiza as trocas de contexto e reduz expressivamente o custo inerente à virtualização.

Desse modo, o *VirtIO* consolida-se como uma solução fundamental para a infraestrutura de virtualização moderna, pois alcança um equilíbrio entre a flexibilidade de uma interface padronizada e o desempenho da paravirtualização.

2.10 DIFERENÇAS ENTRE VIRTUALIZAÇÃO E EMULAÇÃO

Algumas funcionalidades implementadas neste trabalho - devido a limitações de hardware - serão emuladas em vez de virtualizadas. A principal diferença entre esses dois conceitos está na forma em que as operações são executadas.

A emulação é realizada inteiramente por software, reproduzindo o comportamento de uma arquitetura ou dispositivo por meio da interpretação ou tradução de instruções. Entretanto, devido à necessidade de traduzir instruções e simular componentes de hardware, a emulação tende a apresentar menor desempenho em relação a virtualização.

Por outro lado, embora a virtualização também introduza uma camada de software, esta atua primariamente como uma interface de controle e gerenciamento. Isso permite que a maior parte das instruções seja executada diretamente pelo processador físico. Contudo, essa abordagem exige suporte especializado por parte do hardware subjacente, um requisito que nem sempre está disponível em todas as arquiteturas e máquinas.

2.11 HIPERVISORS

Um hipervisor, também conhecido como Monitor de Máquina Virtual (VMM), é uma plataforma que cria e executa máquinas virtuais via virtualização. Segundo Popek e Goldberg (1974) um VMM possuí três características essenciais: primeiro, devem fornecer um ambiente idêntico à máquina original para programas; segundo, os programas executados nesse ambiente, na pior das hipóteses, devem apresentar no máximo pequenas reduções de velocidade que urgem da camada de virtualização; e por último, os VMMs devem deter o contole total sobre os recursos do sistema.

Na visão de (GOLDBERG, 1973), fundamentalmente, existem dois tipos de hipervisores: 1 e 2.

2.11.1 Tipo 1 - Bare Metal

Hipervisores de tipo 1, também conhecidos como bare metal, são aqueles que executam diretamente sobre o hardware físico, sem a necessidade de um sistema operacional hospedeiro intermediário. Por operar diretamente sobre o hardware, esse tipo de hipervisor geralmente apresenta melhor desempenho e menor sobrecarga.

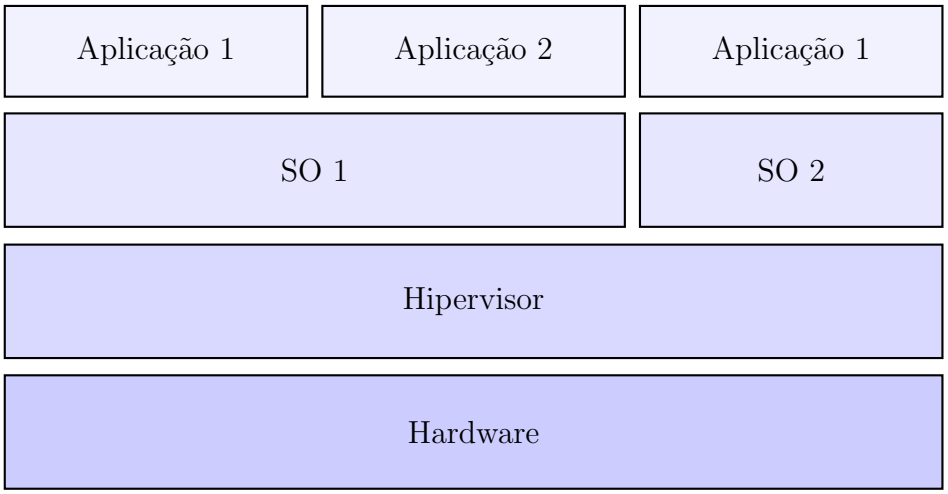


Figura 1 – Hipervisor Tipo 1.

2.11.2 Tipo 2

Diferentemente da solução bare metal, um hipervisor do tipo 2 é executado como um processo dentro de um SO hospedeiro. Porém, apesar de garantir os mesmos recursos do tipo 1, sua arquitetura impõe uma camada intermediária a mais de software, o que restringe a eficiência no acesso aos componentes físicos, gerando uma latência adicional e um menor desempenho.

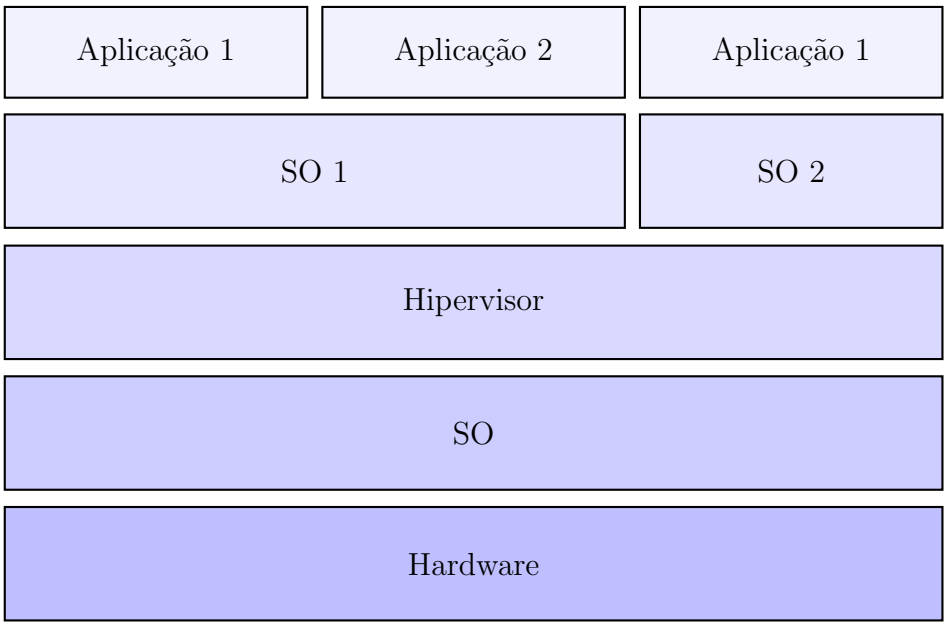


Figura 2 – Hipervisor Tipo 2.

2.12 INTERCEPTAÇÃO E EMULAÇÃO

Do inglês *trap-and-emulate*, interceptação e emulação trata-se de um método utilizado para viabilizar a virtualização em sistemas nos quais o suporte de hardware não é completamente implementado. Nesse modelo, quando a aplicação executada em ambiente virtualizado tenta realizar uma operação que depende de recursos privilegiados, essa operação é interceptada e tratada como uma exceção. O controle é então transferido para um nível de privilégio superior, geralmente o hipervisor, que fica responsável por emular em software o comportamento esperado da instrução ou operação originalmente solicitada.

2.13 RISC-V

A RISC-V é uma Instruction Set Architecture (ISA) baseada nos princípios RISC (Reduced Instruction Set Computer). Diferentemente de arquiteturas proprietárias, a RISC-V possui uma especificação aberta e livre de royalties, o que permite sua utilização tanto em contextos acadêmicos quanto em aplicações comerciais.

Essa ISA define um conjunto básico de instruções, responsável pelas funcionalidades essenciais do processador. No entanto, sua principal característica é a modularidade, permitindo que diferentes extensões sejam incorporadas conforme as necessidades da aplicação. Dessa forma, é possível adaptar a arquitetura para cenários variados, desde sistemas embarcados com restrições de recursos até sistemas de alto desempenho.

2.13.1 Extensões

I Adiciona suporte às instruções básicas de operações com números inteiros. Conforme descrito em (INTERNATIONAL, 2024), essa extensão deve estar presente em qualquer implementação RISC-V, pois define as operações fundamentais de aritmética e manipulação de registradores.

Zicsr Adiciona suporte às instruções de acesso e manipulação de Control and Status Registers (CSRs).

H Adiciona suporte em hardware à virtualização. Porém, apesar de existir, é muito recente e sua implementação em hardware comercial ainda é limitada.

M Adiciona suporte à operações de multiplicação e divisão de números inteiros.

A Adiciona suporte à operações atômicas em memória.

F Adiciona suporte a números em ponto flutuante de precisão simples (32 bits).

D Adiciona suporte a números em ponto flutuante de precisão dupla (64 bits).

C Adiciona suporte à instruções comprimidas de 16 bits.

S Adiciona suporte ao modo supervisor.

G Aglomerado geral de extensões que inclui I, M, A, F, D e C.

Para o escopo deste trabalho, serão usadas apenas as extensões Zicsr, S e G. Os motivos e estratégias adotadas para mitigar e justificar a ausência da extensão H serão discutidas ao longo do texto.

2.13.2 Níveis de Privilégio

Assim como a maior parte das arquiteturas modernas, a RISC-V utiliza um modelo hierárquico de privilégios para garantir a segurança e a estabilidade dos sistemas que funcionam sobre ela. Esses níveis de privilégio determinam quais operações e recursos de hardware podem ser acessados por cada contexto de execução.

Por exemplo, ao executar o SO em um nível mais privilegiado, torna-se possível protegê-lo contra interferências de aplicações executadas em níveis inferiores. Dessa forma, o sistema operacional pode impedir tanto que aplicações comprometam seu funcionamento quanto que interfiram entre si, aumentando a segurança do sistema como um todo.

Essa proteção é normalmente implementada por meio de mecanismos de hardware, como registradores privilegiados e regiões de memória protegidas, cujo acesso é restrito de acordo com o nível de privilégio de cada contexto de execução.

O modelo básico da arquitetura estudada neste documento, conforme mostra a Figura 3, contém três níveis de privilégio.

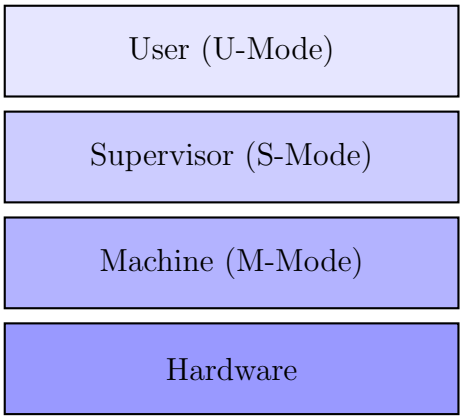


Figura 3 – Níveis de Privilégio RISC-V.

User Mode Nível mais restrito de execução, destinado à operação de aplicações que não devem possuir permissões suficientes para comprometer a integridade, a segurança ou o funcionamento do sistema.

Supervisor Mode Nível utilizado pela maior parte dos SOs modernos, permitindo que eles gerenciem os recursos da máquina e controlem as permissões associadas ao *User Mode*.

Machine Mode Nível mais privilegiado da arquitetura, responsável pelo controle direto do hardware e pela execução de operações críticas de baixo nível. Seus aspectos serão discutidos com maior profundidade ao longo deste texto.

2.13.3 Registradores de Controle

Os CSRs são registradores especiais utilizados para controlar e monitorar o funcionamento do processador. Eles armazenam informações essenciais relacionadas ao estado da máquina, ao tratamento de exceções e interrupções, à configuração de privilégios e ao controle de funcionalidades específicas da arquitetura.

Esses registradores são organizados de forma segregada por nível de privilégio, sendo que cada nível dispõe do seu próprio conjunto de CSRs. Esse isolamento é fundamental para a segurança e a integridade do sistema, pois garante uma política rígida de acessibilidade: um nível de privilégio só pode acessar seus próprios registradores ou aqueles pertencentes a níveis menos privilegiados. Qualquer tentativa de acesso indevido a um CSR de nível inferior resulta em uma exceção de instrução ilegal, impedindo que código menos privilegiado interfira diretamente nas camadas críticas do sistema.

2.14 PREEMPÇÃO E ESCALONAMENTO

Preempção é o mecanismo pelo qual o sistema operacional interrompe temporariamente a execução de uma aplicação — neste contexto, podendo ser uma Máquina Virtual (VM) — para recuperar o controle de um núcleo da CPU. Esse mecanismo é fundamental para a implementação do escalonamento, permitindo que múltiplas aplicações compartilhem um mesmo processador de forma eficiente.

Conforme discutido na seção de máquinas virtuais, cada VM é representada internamente por uma *Thread* no SO hospedeiro. Dessa forma, o escalonamento de máquinas virtuais pode ser tratado como um caso particular do escalonamento de *Threads*.

Nesse contexto, o processo de escalonamento do hipervisor pode ocorrer de duas formas: voluntário ou involuntário.

2.14.1 Escalonamento Involuntário

O escalonamento involuntário ocorre independentemente das ações da *Thread* ou da VM em execução. Nesse caso, o temporizador de hardware gera interrupções periódicas que forçam a preempção da tarefa em execução, transferindo o controle para o kernel do

hipervisor. Ao receber essa interrupção, o escalonador é acionado e avalia qual *Thread* deverá utilizar o núcleo no próximo intervalo de tempo. Caso seja necessário substituir a *Thread* atualmente em execução, o sistema realiza uma troca de contexto entre a *Thread* interrompida e a nova *Thread* selecionada para execução.

2.14.2 Escalonamento Voluntário

O escalonamento voluntário ocorre quando uma *Thread* ou uma VM passa a aguardar a disponibilidade de algum recurso, deixando voluntariamente o processador. Durante esse período de inatividade, outras Threads podem ser executadas até que a entidade bloqueada esteja novamente apta a prosseguir. Esse tipo de escalonamento pode ocorrer em diferentes partes do sistema, sendo um dos casos mais comuns aquele em que uma VM necessita realizar uma operação de IO e precisa aguardar sua vez para acessar um controlador de hardware.

Outra forma de escalonamento voluntário ocorre quando a VM em execução entra em estado de idle. Nesse cenário, o hipervisor deve identificar essa condição e verificar a existência de outras VMs prontas para execução. Apenas na ausência de outras demandas, o sistema permite que a CPU entre efetivamente em estado de idle.

Entretanto, a detecção do momento em que uma VM passa para o estado de idle não é trivial, e exige suporte específico da arquitetura para que essa transição seja identificada.

3 MATERIAIS UTILIZADOS

Para a realização deste trabalho foi utilizada placa - Single Board Computer (SBC) - VisionFive 2. Considerando os objetivos propostos, as principais características de hardware relevantes para este estudo são apresentadas a seguir:

- System on a Chip (SoC): StarFive JH7110;
- DRAM: 8 GB LPDDR4;
- Interface Gigabit Ethernet: Synopsys DesignWare DWMAC 5.20;
- PMIC: X-Powers AXP15060.

O SoC StarFive JH7110 opera com frequência de até 1,5 GHz e possui uma arquitetura heterogênea composta por seis núcleos de processamento, distribuídos da seguinte forma:

- 4 núcleos U74 de 64 bits, compatíveis com a arquitetura RISC-V RV64GC;
- 1 núcleo S7 de 64 bits, compatível com a arquitetura RISC-V RV64IMAC;
- 1 núcleo E24 de 32 bits, compatível com a arquitetura RISC-V RV32IMAFCB.

Cabe destacar que todos os experimentos realizados neste trabalho foram conduzidos utilizando essa placa. Além disso, foram considerados apenas os quatro núcleos U74, uma vez que os núcleos S7 e E24 não oferecem suporte ao modo supervisor, requisito indispensável para a implementação dos mecanismos de virtualização aqui analisados.

4 METODOLOGIA

4.1 VIRTUALIZAÇÃO BASEADA *MACHINE MODE*

Originalmente, o *Machine Mode* (M-mode) foi projetado para ser o nível de privilégio mais elevado da arquitetura RISC-V, sendo responsável pela execução do firmware base e pelo controle direto dos recursos de hardware. Sua função primária consiste na inicialização da plataforma e na configuração dos subsistemas fundamentais antes da transferência de controle para o SO hospedeiro.

Dada a natureza restritiva do acesso a determinados recursos de hardware — muitas vezes limitados exclusivamente ao M-mode —, a arquitetura define camadas de abstração como a Supervisor Binary Interface (SBI). Essa interface atua como uma ponte padronizada e segura, permitindo que níveis de privilégio inferiores solicitem serviços privilegiados ao firmware residente no M-mode.

No contexto deste trabalho, o M-mode é utilizado de forma não convencional. Para isso, deixamos de usar o M-Mode como um simples SBI e passamos a usalo para dar suporte a implementação de um hipervisor completo.

Essa abordagem explora o fato de que o M-mode possui autoridade irrestrita sobre todos os demais níveis de privilégio, permitindo interceptar, controlar e gerenciar a execução dos sistemas convidados. Para viabilizar esse controle, o hipervisor utiliza os recursos exclusivos disponibilizados nesse modo, incluindo seu conjunto dedicado de CSRs, além de mecanismos próprios para tratamento de interrupções e gerenciamento de temporização. Dessa forma, o M-mode fornece a soberania necessária para reproduzir, por software, e as vezes até em hardware, funcionalidades que normalmente só seriam oferecidas por hardwares especializados compatíveis com as extensões de virtualização.

5 IMPLEMENTAÇÃO

Neste capítulo, é apresentada a implementação de todo o software desenvolvido neste trabalho.

5.1 ESTRUTURA DE PASTAS

O projeto foi organizado de forma modular, com a separação dos componentes de acordo com sua hierarquia e responsabilidade dentro do sistema. Essa estrutura facilita a manutenção, a evolução do código e a integração entre as diferentes camadas da aplicação, além de contribuir para a organização e otimização do processo de compilação.

```
application/
architecture/
build/
include/
machine/
src/
tools/
```

- *application/*: contém aplicações do usuário que podem ser embutidas na inicialização do sistema;
- *architecture/*: reúne o código genérico relacionado à arquitetura da máquina, independente de implementações específicas;
- *build/*: diretório gerado automaticamente durante a compilação, contendo arquivos intermediários e executáveis;
- *include/*: armazena os arquivos de cabeçalho em C++, responsáveis por declarações de interfaces e definições compartilhadas;
- *machine/*: contém o código específico de uma determinada máquina, incluindo implementações dependentes de hardware ou plataforma;
- *src/*: concentra o código-fonte principal em C++, com a implementação das funcionalidades do sistema;
- *tools/*: reúne utilitários e ferramentas auxiliares utilizados no processo de compilação, depuração e suporte ao sistema;

5.2 INICIALIZAÇÃO

A inicialização do hipervisor ocorre imediatamente após o controle ser transferido pelo bootloader - U-Boot, responsável por carregar o binário do hipervisor na memória e transferir a execução para seu ponto de entrada.

O ponto de entrada corresponde a uma função dependente da arquitetura, responsável por realizar a inicialização das estruturas mais básicas do sistema. Nessa etapa, são configurados os elementos fundamentais de execução da CPU e do ambiente de hardware, permitindo a transição gradual para camadas superiores do sistema.

A partir dessa função inicial, o controle é transferido sequencialmente para módulos de maior nível de abstração, até que o sistema alcance o ponto de entrada do escalonador. O fluxo padrão de inicialização segue a seguinte sequência:

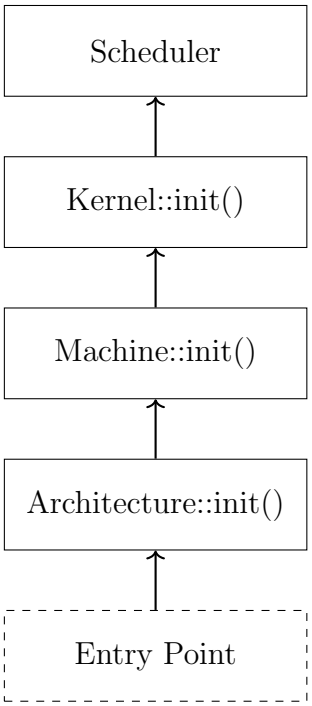


Figura 4 – Fluxo Geral de Inicialização do Hipervisor.

5.2.1 RISC-V

O núcleo da plataforma de virtualização é estabelecido nesta etapa. O papel principal dessa camada de inicialização consiste em instalar os tratadores de interrupções para a emulação de dispositivos de *Memory-Mapped I/O* (MMIO) e para o compartilhamento de interrupções entre a máquina hospedeira e os ambientes virtualizados.

A classe responsável por configurar esse vetor de interrupções é a Hypervisor Interrupt Controller (HIC). O fluxo completo de inicialização é detalhado pelo seguinte diagrama:

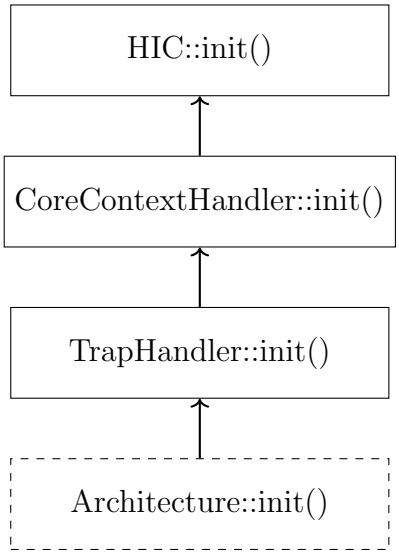


Figura 5 – Fluxo Geral de Inicialização da Arquitetura RISC-V.

5.2.2 Máquina

Subjacente à arquitetura, a inicialização da máquina lida com as especificidades da plataforma alvo. Nesta etapa, a rotina *Machine::init()* mapeia os periféricos essenciais para a operação global do sistema, tais como os controladores de console para a emissão de *logs* e barramentos de comunicação específicos da placa de desenvolvimento utilizada.

5.2.3 Kernel

Nesta etapa da inicialização do sistema, são executadas as rotinas de configuração dos objetos gerais que serão compartilhados e gerenciados pelo sistema operacional. Esse processo abrange a inicialização de subsistemas críticos, tais como o gerenciamento de memória e as estruturas de dados necessárias para o controle e a organização de tarefas.

Após isso, o fluxo de inicialização atinge o seu estágio final, momento no qual o sistema transfere o controle definitivo da CPU para o escalonador, dando início à execução das aplicações do usuário.

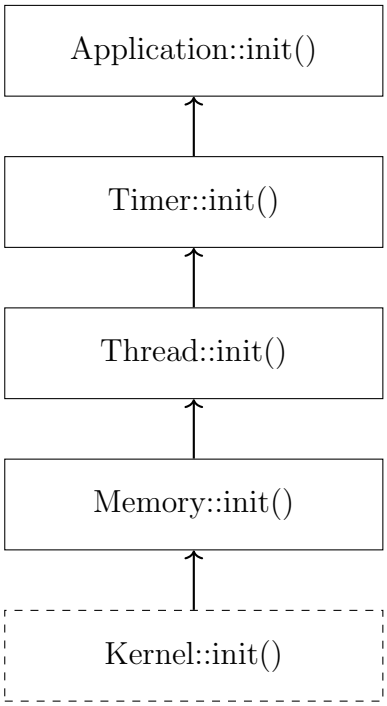


Figura 6 – Fluxo Geral de Inicialização do Kernel.

5.3 MÁQUINAS VIRTUAIS

A classe *VirtualMachine*, apresentada na Figura 7, é a interface de controle das VMs em C++ dentro do sistema, responsável por abstrair as funcionalidades e componentes fundamentais que toda VM deve possuir.

Por padrão, toda VM deve receber, no mínimo, um *chunk* estático de memória que delimita todo o escopo de memória da máquina. Esse *chunk* é passado para um *VirtualMemoryManager*, que será utilizado por diversos componentes da camada de virtualização para verificar se uma máquina possui ou não acesso a uma determinada região de memória.

Além disso, toda VM deve implementar, pelo menos, três funções básicas: *read*, *write* e *interrupt*.

As funções *read* e *write* são essenciais para a emulação de dispositivos virtuais. Sempre que uma VM tenta realizar um acesso a um endereço de memória não permitido, o hypervisor, antes de encerrar a execução da VM, delega a operação para essas funções e aguarda o resultado. Dentro delas, o bloco de controle da VM decide se a operação é válida. Em caso afirmativo, a operação é emulada e um retorno positivo é enviado ao chamador. Caso contrário, a operação é rejeitada e o acesso é tratado como um erro crítico para a VM.

A função *interrupt* é responsável por encaminhar interrupções da máquina hospedeira para a máquina convidada. Essa função é predominantemente utilizada por disposi-

tivos virtuais para notificar o convidado sobre eventos específicos ocorridos no controlador.

VirtualMachine
- memory_ : VirtualMemoryManager
VirtualMachine(chunk : Chunk) + interrupt(id : uint32_t) : void + memory() : VirtualMemoryManager + read(address : uintptr_t, destination : * uintmax_t) : bool + write(address : uintptr_t, source : uintmax_t) : bool

Figura 7 – Diagrama de Classe: VirtualMachine.

5.3.1 Inicialização

Por definição, uma máquina virtual consiste essencialmente em uma *Thread* do sistema associada a uma estrutura de virtualização. A criação de uma VM ocorre quando uma *Thread* instancia um objeto do tipo *VirtualMachine*. A partir desse momento, a *Thread* passa a operar dentro de um ambiente virtualizado, tornando-se sujeita aos mecanismos de isolamento, gerenciamento de memória e emulação de dispositivos providos pelo hipervisor. Um melhor detalhamento desse processo depende de implementações específicas, uma vez que esse objeto constitui apenas uma interface.

5.3.2 *GenericVirtualMachine*

A classe *GenericVirtualMachine* implementa a interface *VirtualMachine* com o objetivo de simplificar a criação e a instanciação de máquinas virtuais e de seus respectivos periféricos. Estruturada como um *template* variádico, ela permite receber um número arbitrário de tipos como parâmetros, os quais são instanciados, mapeados e inicializados automaticamente como dispositivos internos da máquina virtual.

Um aspecto importante dessa modelagem é que os controladores de interrupção também são tratados como dispositivos. Dessa forma, eles devem ser fornecidos explicitamente como parâmetros do *template*, possibilitando sua integração desde o momento da inicialização da VM.

Assim como toda implementação de *VirtualMachine*, o construtor da classe *GenericVirtualMachine* recebe um *Chunk* que define o espaço de endereçamento da máquina virtual. Esse bloco de memória é utilizado como base para a configuração do ambiente virtualizado e para o gerenciamento dos dispositivos associados à instância.

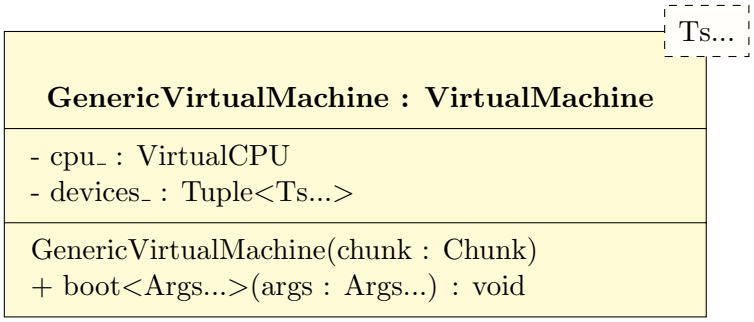


Figura 8 – Diagrama de Classe: GenericVirtualMachine.

5.3.2.1 Inicialização

Durante sua inicialização, a GenericVirtualMachine cria e configura uma instância de VirtualCPU e realiza a instanciação e inicialização de todos os dispositivos especificados. O fluxo de execução desse processo é apresentado a seguir:

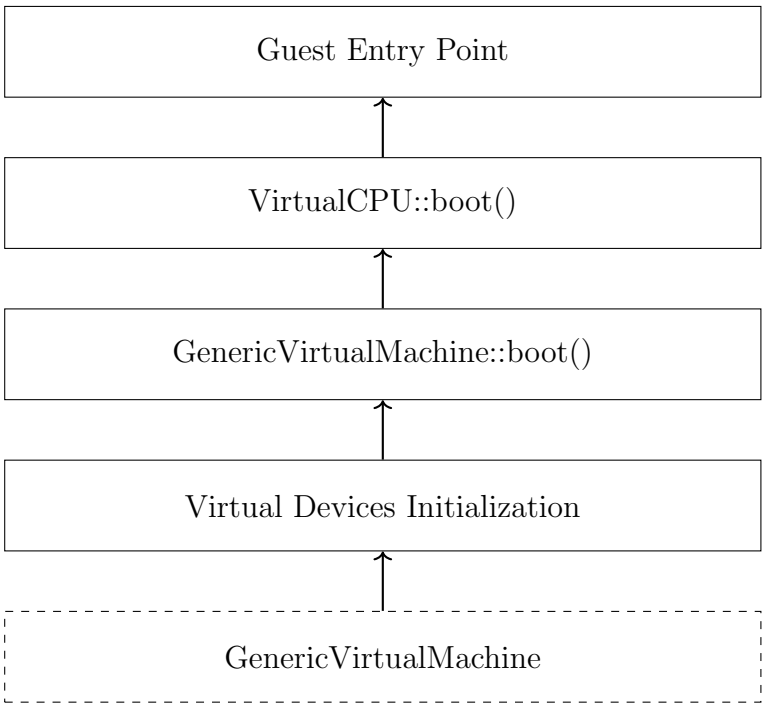


Figura 9 – Fluxo Geral de Inicialização de uma *GenericVirtualMachine*.

5.4 CPUS VIRTUAIS

A classe *VirtualCPU* representa a unidade de básica de processamento de uma VM, sendo responsável pelo encaminhamento de interrupções, pela emulação de exceções

e pelo gerenciamento da execução dos SOs convidados. Como sua implementação depende da arquitetura da máquina hospedeira, detalhes específicos são apresentados nas seções correspondentes a cada arquitetura.

5.4.1 RISC-V

O diagrama de classes de *VirtualCPU* - para a arquitetura RISC-V - é ilustrado na Figura 10. Para viabilizar a transferência de eventos para a máquina virtual, a classe mantém uma referência direta para a VM à qual está associada. Ademais, esta implementação incorpora o registrador *mtimecmp* emulado para os mecanismos de temporização, uma vez que existe apenas um registrador físico por máquina hospedeira. A classe também possui o atributo estático *current_*, mapeado por núcleo, que armazena um ponteiro para a instância de *VirtualCPU* em execução em cada núcleo físico.

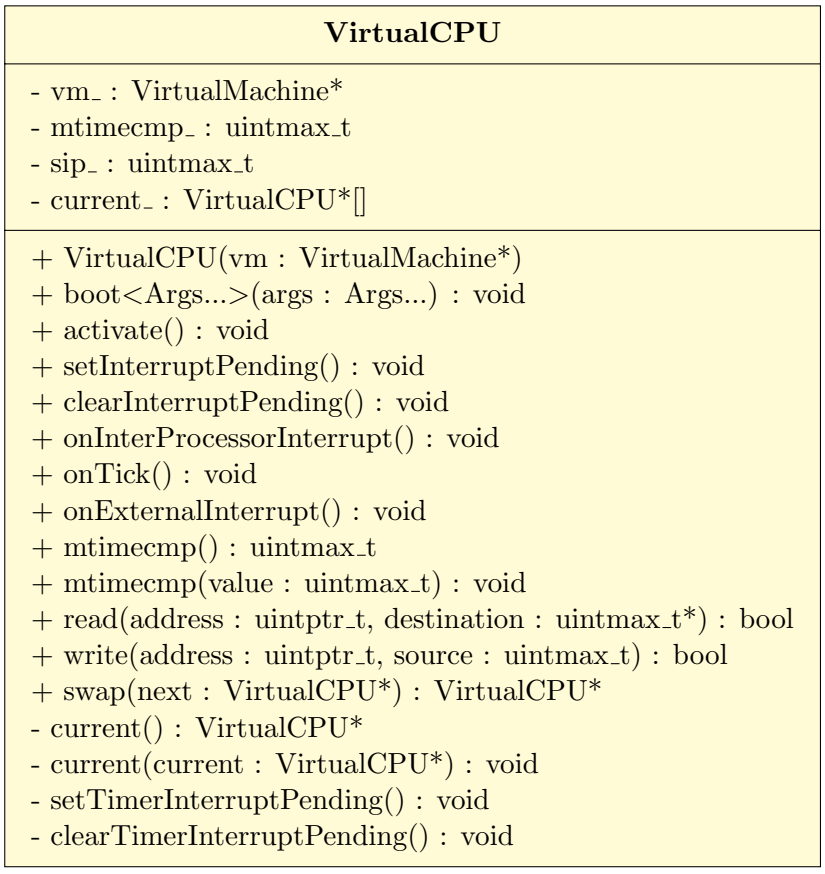


Figura 10 – Diagrama de classe da *VirtualCPU*.

5.4.1.1 Virtualização de Interrupções Temporais

Interrupções de temporização são virtualizadas por meio do registrador *mtimecmp* emulado em cada instância de *VirtualCPU*. O SO convidado atualiza o valor do seu

mtimecmp por meio de uma *hypercall*. A partir dessa escrita, e enquanto a respectiva *VirtualCPU* estiver em execução, o hipervisor verifica a cada *tick* do sistema hospedeiro se o *mtimecmp* do convidado foi atingido ou superado pelo registrador de tempo global. Caso essa condição seja atendida, uma interrupção é injetada no bit Supervisor Timer Interrupt (STI) contido no CSR Supervisor Interrupt Pending (SIP) do convidado.

5.4.1.2 Virtualização de Interrupções Externas

Interrupções externas não são controladas diretamente pela *VirtualCPU*; tal classe fica responsável apenas por injetá-las no sistema convidado. Para isso, a *VirtualCPU* expõe uma interface para que outros dispositivos sejam capazes de gerar esses eventos. Tais eventos são injetados por meio da sinalização do bit Supervisor External Interrupt (SEI) no CSR SIP do convidado.

5.4.1.3 Inicialização

A inicialização de uma *VirtualCPU* RISC-V consiste na preparação do ambiente de execução e na subsequente transição para o ponto de entrada da VM já em modo supervisor; uma vez que a maioria dos SOs convidados - como o Linux - esperam inicializar nesse modo.

O fluxo ordenado das etapas executadas durante esse processo de inicialização está ilustrado na Figura 11:

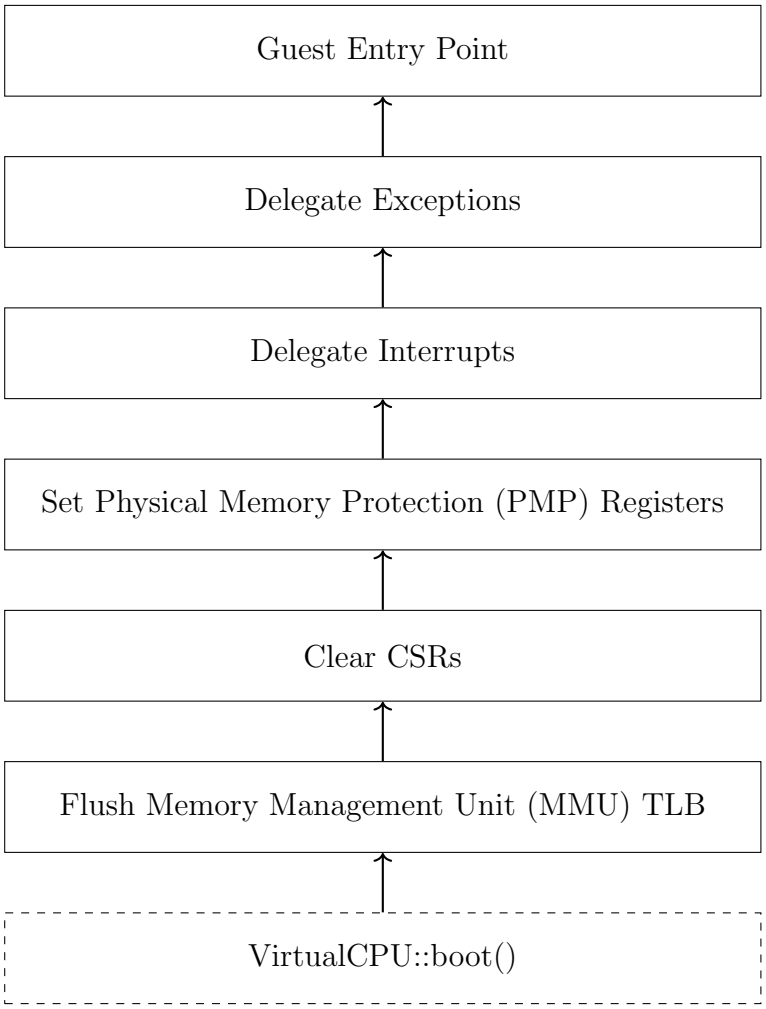


Figura 11 – Fluxo Geral de Inicialização de uma *VirtualCPU* RISC-V.

5.5 TROCA DE CONTEXTO ENTRE MÁQUINAS VIRTUAIS

A troca de contexto refere-se ao processo de interromper a execução de uma máquina virtual em andamento, salvando seu estado atual e restaurando o contexto de outra máquina virtual para que esta possa ser executada no processador.

Essa é uma das etapas mais críticas de um hipervisor, pois a corrupção do contexto de uma VM pode praticamente inutilizá-la.

5.5.1 RISC-V

Para realizar essa operação na RISC-V, o processo foi dividido em duas etapas: a manipulação do contexto da CPU — que inclui estruturas internas do próprio hipervisor associadas a cada VM — e a manipulação do contexto no escopo da máquina virtual, responsável por elementos específicos necessários ao SO convidado.

O contexto de CPU concentra-se no salvamento dos registradores gerais e dos específicos do modo *machine*, bem como na gestão em nível de *Thread*, que constitui a unidade básica de execução do hipervisor.

Por sua vez, o contexto da VM é responsável pelo salvamento de registradores que extrapolam o escopo do hipervisor, incluindo os registradores do modo *supervisor*, utilizados exclusivamente pelas VMs. Além disso, esse contexto gerencia a troca de espaços de endereçamento, a delegação de interrupções e a invalidação do cache da MMU.

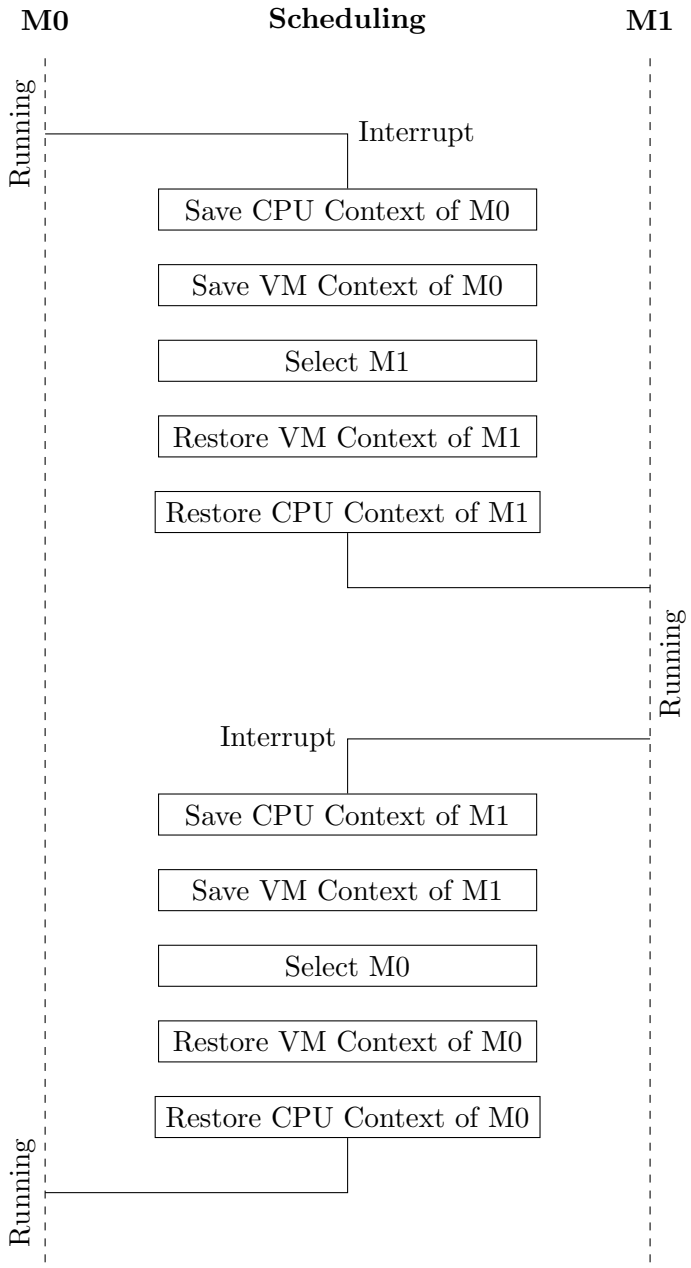


Figura 12 – Troca de Contexto Entre Duas VMs

5.6 CAMADA DE EMULAÇÃO

Os recursos que não podem ser virtualizados neste trabalho são, por consequência, emulados. Como discutido anteriormente, a emulação apresenta um custo computacional superior ao da virtualização. Em contrapartida, oferece maior versatilidade, pois sua execução ocorre inteiramente em software.

A camada de emulação proposta baseia-se na técnica de *trap-and-emulate*. Dessa forma, toda operação emulada é precedida por uma exceção e, em algum grau, depende das características da arquitetura subjacente. Quando uma exceção ocorre, a camada de emulação verifica se ela foi causada por uma tentativa de acesso a um recurso passível de emulação. Em caso afirmativo, a operação correspondente é realizada em software. Caso contrário, a exceção é tratada como um erro crítico, podendo resultar na desativação completa da VM.

5.6.1 RISC-V

Em RISC-V, para viabilizar esse recurso, foi introduzida uma camada intermediária entre o mecanismo de tratamento de erros críticos do sistema e a exceção originalmente gerada. Essa camada permite identificar situações passíveis de emulação antes que sejam encaminhadas ao tratador de erros críticos. Atualmente, as seguintes exceções podem representar erros emuláveis:

- *Illegal Instruction*
- *Load Access Fault*
- *Store Access Fault*

5.6.1.1 *Illegal Instruction*

A exceção *Illegal Instruction* é gerada quando o processador encontra uma instrução não suportada, ou não permitida em determinado contexto. Esse mecanismo é amplamente utilizado para emular extensões ausentes, possibilitando que a semântica da instrução seja reproduzida por software de forma transparente para o sistema em execução. Neste trabalho, usaremos esse mecanismo para detectar leituras no CSR *mtime*, alterações no estado da *Floating-Point Unit* (FPU) e para detecção de *idle* que será melhor explicada na seção de escalonamento.

5.6.1.2 *Load Access Fault*

A exceção *Load Access Fault* é gerada durante uma operação de leitura em uma região de memória que não pode ser acessada diretamente pelo contexto de execução

atual. No mecanismo de emulação, essa exceção permite interceptar a tentativa de leitura e fornecer os dados por meio de uma rotina de software, sendo utilizada principalmente na emulação de dispositivos MMIO. Quando a exceção é disparada, o hipervisor verifica inicialmente se o endereço acessado pertence a algum dispositivo MMIO associado à VM. Caso o endereço corresponda a um dispositivo emulado, são realizadas validações adicionais para determinar se a operação solicitada é suportada e válida para aquele dispositivo. Somente após a conclusão dessas verificações a operação de leitura é efetivamente emulada e seu resultado retornado à VM. O fluxo de tratamento dessa exceção no sistema é ilustrado na Figura 13.

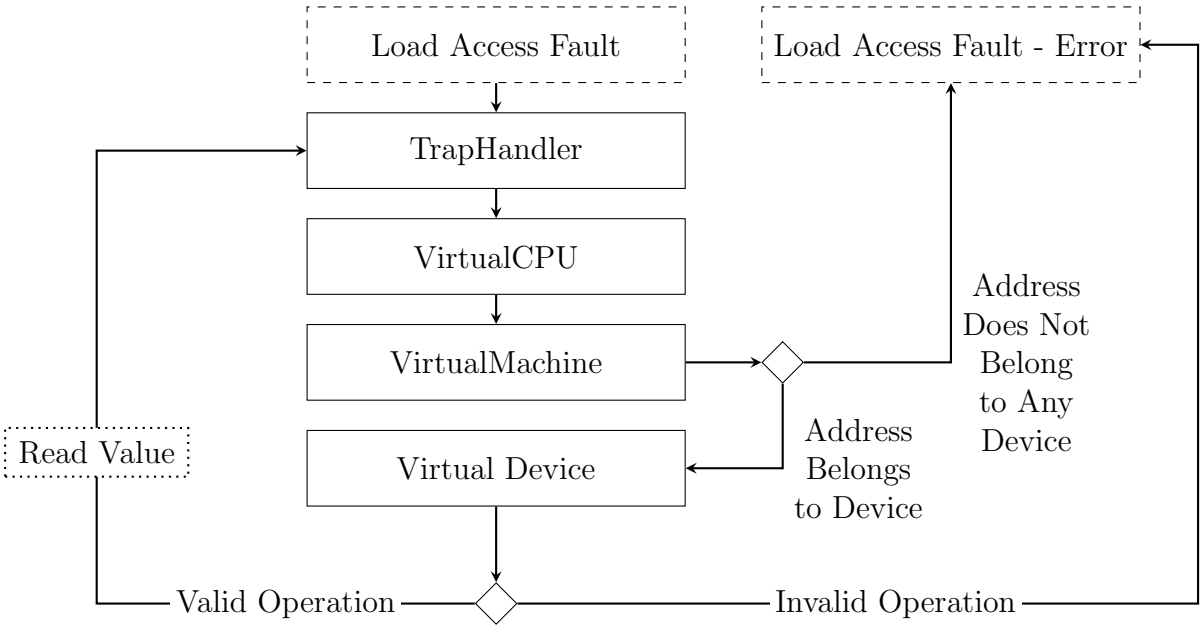


Figura 13 – Fluxo de Tratamento de um *Load Access Fault* - RISC-V.

5.6.1.3 *Store Access Fault*

A exceção *Store Access Fault* é gerada durante uma operação de escrita em uma região de memória que não pode ser acessada diretamente pelo contexto de execução atual. No mecanismo de emulação, essa exceção permite interceptar a tentativa de escrita e tratar a operação por meio de uma rotina de software, sendo utilizada também na emulação de dispositivos MMIO. Quando a exceção é disparada, o hipervisor verifica inicialmente se o endereço acessado pertence a algum dispositivo MMIO associado à VM. Caso o endereço corresponda a um dispositivo emulado, são realizadas validações adicionais para determinar se a operação solicitada é suportada e válida para aquele dispositivo. Somente após a conclusão dessas verificações a operação de escrita é efetivamente emulada. O fluxo de tratamento dessa exceção no sistema é ilustrado na Figura 14.

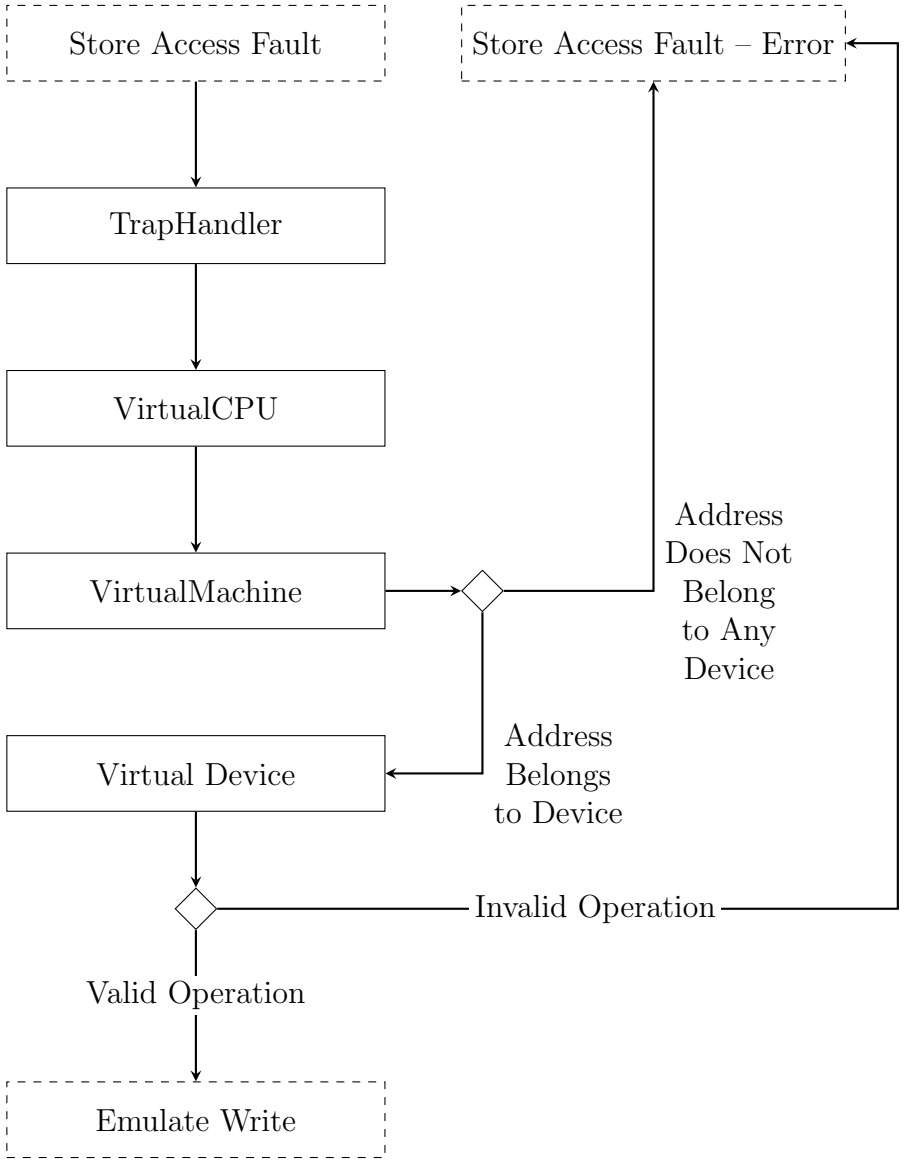


Figura 14 – Fluxo de Tratamento de um *Store Access Fault* – RISC-V.

5.7 DETECÇÃO E TRATAMENTO DE VMs EM IDLE NA RISC-V

Em plataformas RISC-V, existe uma instrução específica - *Wait For Interrupt* (WFI), utilizada para colocar um núcleo da CPU em estado de *idle*. Ao executá-la, o processador suspende a execução normal até a ocorrência de uma interrupção, reduzindo o consumo de energia enquanto não há tarefas a serem processadas.

Em ambientes virtualizados, é necessário que o hipervisor intercepte essa instrução para gerenciar corretamente o comportamento dos sistemas convidados.

Para isso, a arquitetura disponibiliza um mecanismo de controle de acesso a essa instrução por meio do bit 21 (*Timeout Wait*) do CSR *mstatus*.

Esse bit de configuração permite tratar a instrução WFI como uma *illegal instruction*. Ou seja, quando esse bit está ativado, a execução de uma WFI não resulta diretamente no estado de espera do processador; em vez disso, uma exceção é gerada. Esta exceção é então detectada pelo hipervisor, que - em seguida - faz com que essa VM se afaste do controle da CPU, permitindo que outra ocupe seu lugar. No fim, somente se nenhuma outra *Thread* estiver disponível, a CPU entrará verdadeiramente em *idle*.

6 RESULTADOS

6.1 PRIMEIRO EXPERIMENTO - INTERFERÊNCIA VIA CACHE

O primeiro experimento tem como objetivo avaliar o comportamento e a capacidade de isolamento do hipervisor sob condições de contenção de recursos. Para isso, utilizam-se duas instâncias de VMs executando o SO EPOS, configuradas com perfis de carga distintos e concorrentes. As tarefas foram programadas para execução periódica, permitindo observar a eficácia do hipervisor no gerenciamento e na alternância entre fluxos de trabalho com diferentes intensidades e frequências.

- VM 1 - EPOS com alta demanda de CPU: Esta instância simula uma carga intensiva de processamento por meio da execução de uma versão iterativa do cálculo da sequência de Fibonacci, conforme detalhado no Algoritmo 1. A VM é configurada para disparar esse algoritmo periodicamente a cada 5.000 µs.

Algorithm 1 Fibonacci Iterativo

```
1:  procedure FIBONACCI(number ← 100000)
2:      a ← 0
3:      b ← 1
4:      fib ← 0
5:      for i ← 0 to number − 1 do
6:          fib ← a + b
7:          a ← b
8:          b ← fib
9:      end for
10: end procedure
```

- VM 2 - EPOS com alta demanda de memória: Esta instância estressa o subsistema de memória por meio da aplicação iterativa de uma operação de convolução bidimensional sobre uma imagem sintética de 512×512 pixels, utilizando um kernel de tamanho 3×3. A implementação deste fluxo de trabalho está descrita no Algoritmo 2. Essa VM executa a tarefa periodicamente a cada 100.000 µs.

Algorithm 2 Convolução Bidimensional

```
1: procedure CONVOLUTION
2:     Matrix A[512][512]
3:     Matrix C[512][512]
4:     Kernel K[3][3]
5:     for Iteration ← 1 to 5 do
6:         for i ← 1 to 510 do
7:             for j ← 1 to 510 do
8:                 sum ← 0
9:                 for x ← −1 to 1 do
10:                    for y ← −1 to 1 do
11:                        sum ← sum + A[i + x][j + y] × K[x + 1][y + 1]
12:                    end for
13:                end for
14:                C[i][j] ← sum
15:            end for
16:        end for
17:    end for
18: end procedure
```

Os tempos de execução de cada iteração das aplicações executadas nas VMs ao longo de um intervalo de cinco minutos são apresentados nas Figuras 15, 16, 17, 18, 19 e 20. Cada gráfico mostra a evolução temporal do tempo de execução das tarefas do EPOS sob diferentes cenários de compartilhamento de recursos entre máquinas virtuais.

As Figuras 15 e 16 apresentam os resultados obtidos quando apenas a VM contendo o EPOS está ativa. Nessa configuração observa-se um comportamento relativamente estável, com pequenas oscilações atribuídas ao escalonamento normal do hipervisor. Os tempos de execução das cargas baseadas em CPU e memória permanecem próximos aos valores de referência, caracterizando o desempenho esperado na ausência de interferência externa.

Nas Figuras 17 e 18 é introduzida uma segunda VM executando um Linux sem carga computacional. Embora exista compartilhamento dos núcleos físicos entre as máquinas virtuais, o impacto observado sobre os tempos de execução do EPOS é reduzido. As variações permanecem próximas às observadas no cenário de referência.

Por fim, as Figuras 19 e 20 mostram o comportamento do sistema quando a VM com Linux executa uma carga intensiva de memória, projetada para gerar elevado tráfego no subsistema de memória e frequentes invalidações das linhas de cache compartilhadas entre os núcleos. Nesse cenário, observa-se um aumento expressivo nos tempos de execução da aplicação do EPOS com carga baseada em memória. O efeito torna-se evidente a partir do momento em que a carga concorrente é iniciada, indicando uma degradação diretamente associada à contenção dos recursos de memória e à redução da efetividade do cache.

Em contrapartida, as aplicações com carga predominantemente computacional

apresentam apenas pequenas variações em seus tempos de execução, mesmo diante da atividade intensiva de memória da VM Linux. Esse comportamento sugere a existência de um mecanismo significativo de interferência entre as máquinas virtuais associado ao compartilhamento da hierarquia de cache. Nesse contexto, como a quantidade de memória alocada para cada VM é elevada, estratégias convencionais de particionamento de memória tendem a ser insuficientes para mitigar esse efeito. Assim, soluções efetivas dependem diretamente da capacidade de isolamento provida por controladores específicos de cache — por exemplo, mecanismos de particionamento ou alocação de vias — ou, em cenários experimentais mais extremos, da desativação completa dos níveis de cache compartilhados, de modo a eliminar a fonte de interferência observada.

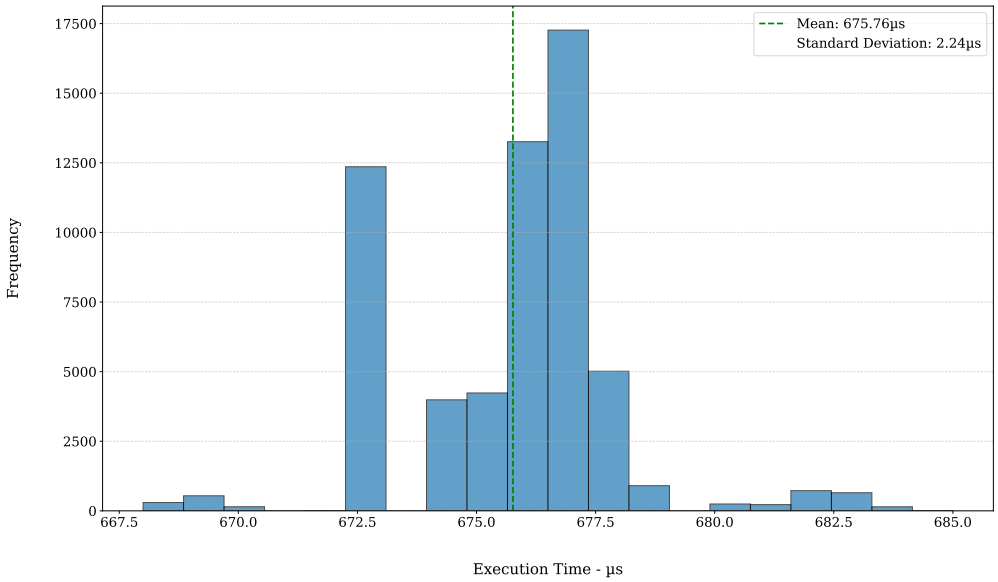


Figura 15 – Tempos de Execução Das Iterações de Uma VM Com Carga em CPU - Sem Linux.

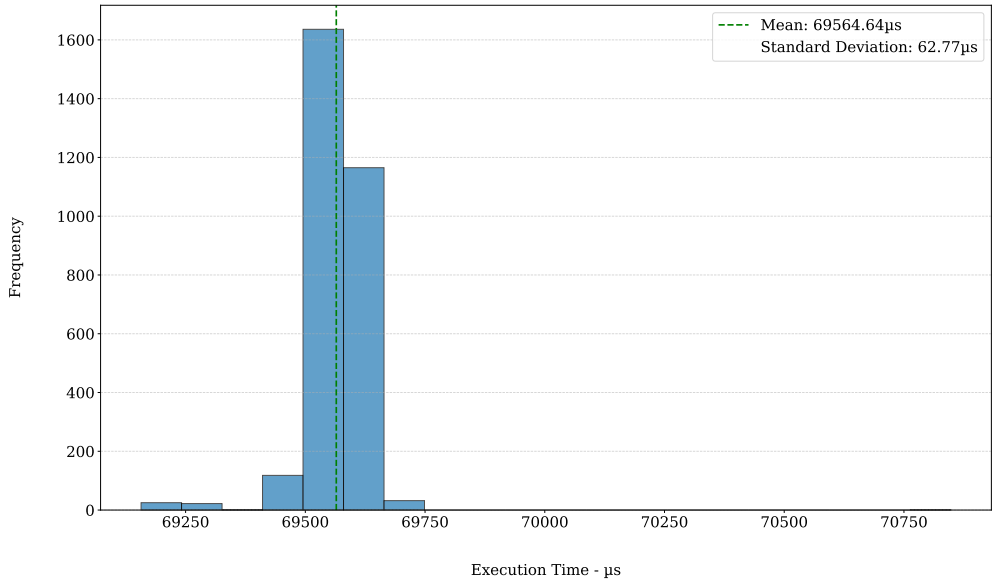


Figura 16 – Tempos de Execução Das Iterações de Uma VM Com Carga em Memória - Sem Linux.

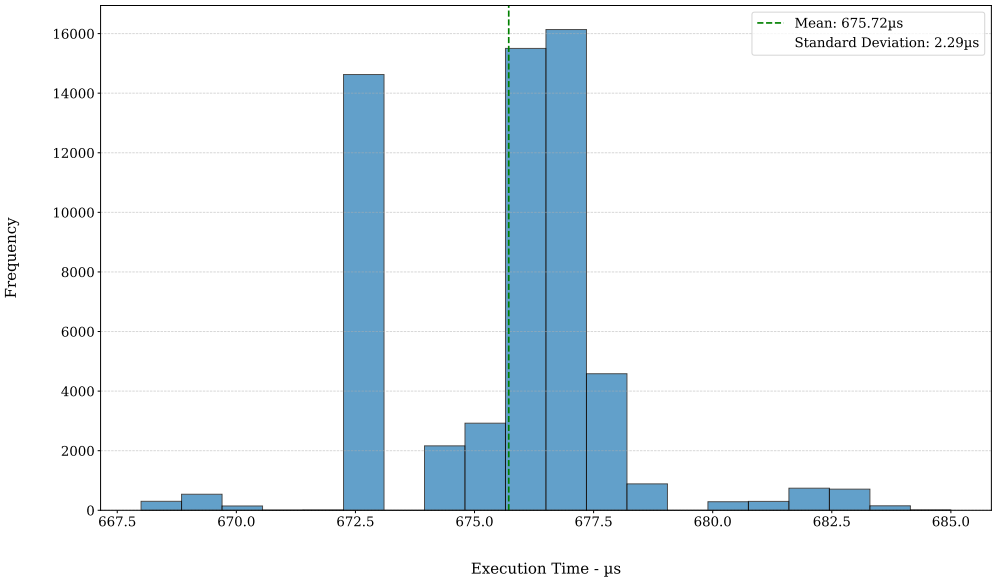


Figura 17 – Tempos de Execução Das Iterações de Uma VM Com Carga em CPU - Com Linux.

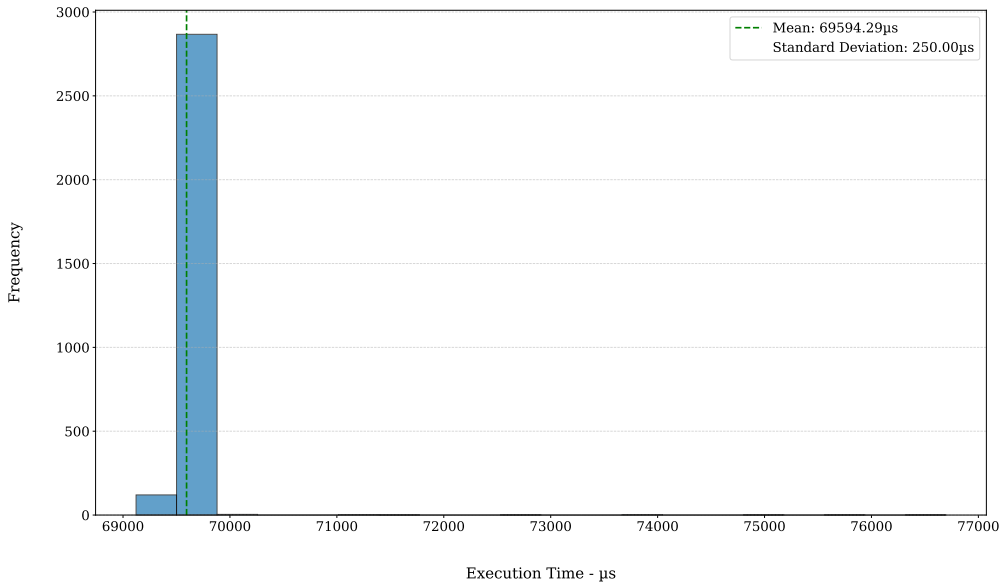


Figura 18 – Tempos de Execução Das Iterações de Uma VM Com Carga em Memória - Com Linux.

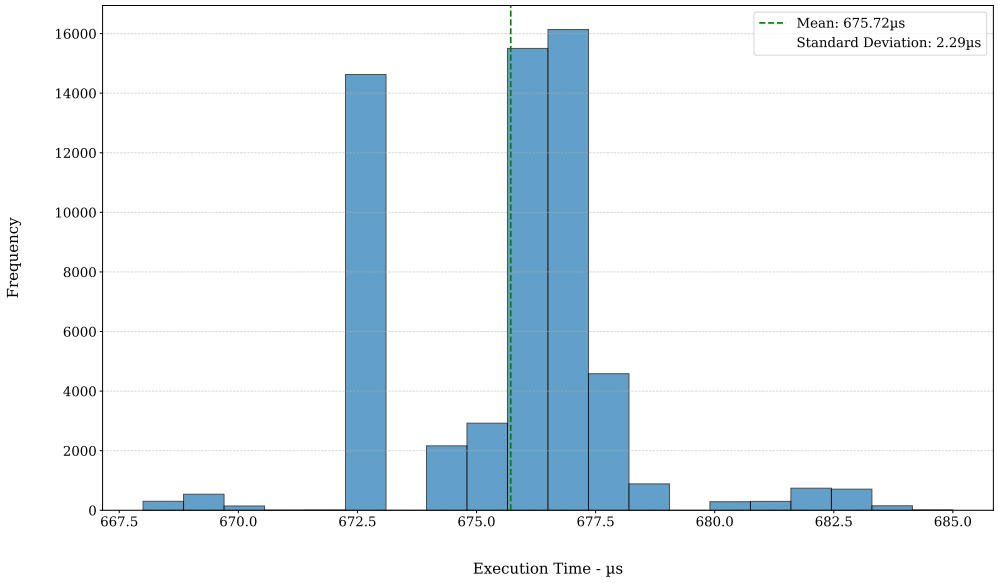


Figura 19 – Tempos de Execução Das Iterações de Uma VM Com Carga em CPU - Com Linux Invalidando Cache.

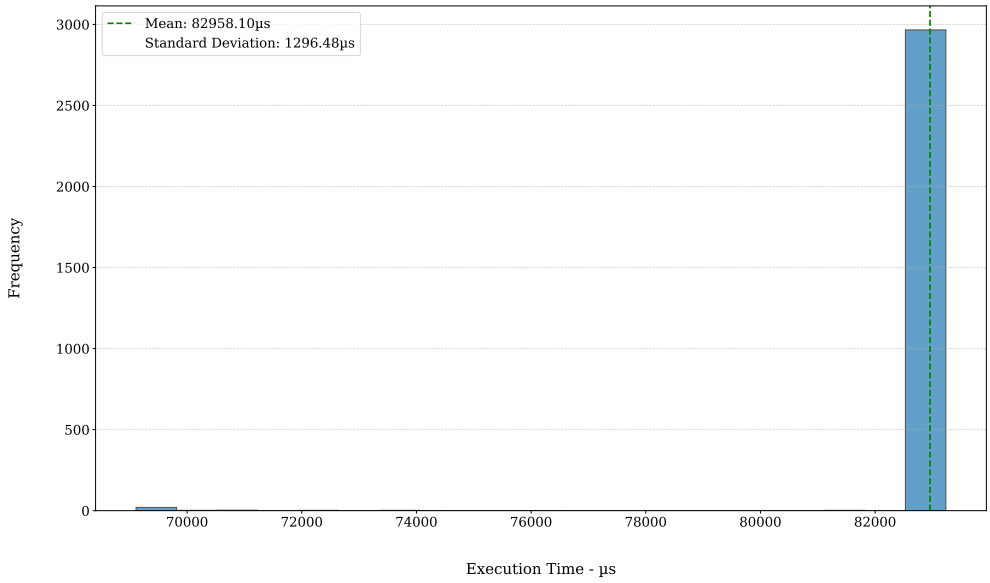


Figura 20 – Tempos de Execução Das Iterações de Uma VM Com Carga em Memória - Com Linux Invalidando Cache.

REFERÊNCIAS

FRöHLICH, A.; SCHRöEDER-PREIKSCHAT, W. High performance application-oriented operating systems - the epos approach. In: *Anais do XI Symposium on Computer Architecture and High Performance Computing*. Porto Alegre, RS, Brasil: SBC, 1999. p. 3–9. ISSN 0000-0000. Disponível em: ⟨https://sol.sbc.org.br/index.php/sbac-pad/article/view/19765⟩.

GOLDBERG, R. P. Architectural principles for virtual computer systems. In: . [s.n.], 1973. Disponível em: ⟨https://apps.dtic.mil/sti/tr/pdf/AD0772809.pdf⟩.

INTERNATIONAL, R.-V. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. 2024. ⟨https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf⟩.

POPEK, G. J.; GOLDBERG, R. P. Formal requirements for virtualizable third generation architectures. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 17, n. 7, p. 412–421, jul. 1974. ISSN 0001-0782. Disponível em: ⟨https://doi.org/10.1145/361011.361073⟩.

RUSSELL, R. virtio: towards a de-facto standard for virtual i/o devices. *SIGOPS Oper. Syst. Rev.*, Association for Computing Machinery, New York, NY, USA, v. 42, n. 5, p. 95–103, jul. 2008. ISSN 0163-5980. Disponível em: ⟨https://doi.org/10.1145/1400097.1400108⟩.